

Browser CI

The three authorization assertions Phase 1 left resting on manual verification are now executable in CI. A deliberately small change: six files, none of them application code.

Branch **claude/create-app-repository-1cxsih**

Head **136d330**

1 commit

CI run **#9** • all three jobs green

App code changed **none**

1

What this is, and how long it deserves

Phase 1's packet named one gap plainly: the most security-relevant behaviour in the running application — a signed-in *ordinary member* being refused an admin route — had no automated proof anywhere. Its only evidence was that someone had run `curl` by hand. A refactor that broke it would have shipped green.

This milestone closes that. It adds three browser tests and a CI job to run them. It is not the start of a UI test suite, and adding a fourth test should be a decision rather than a habit.

SUGGESTED REVIEW TIME: 20-30 MINUTES

Read `apps/web/e2e/authorization.spec.ts` (164 lines) and the mutation table in section 4.

Everything else here is context. **No application code changed** — the diff is CI, a package manifest, gitignore, and the new test files — which is the point: the tests had to fit the application as it stands, not the other way round.

The three assertions

Each runs against a production build served by `next start`, driven by a real browser, over a real session cookie.

#	ASSERTION	WHAT IT ACTUALLY CHECKS
1	Unauthenticated → 401	All three protected routes — <code>/api/v1/me</code> , <code>/api/v1/wallet</code> , <code>/api/v1/admin/ledger</code> — refuse with the stable code <code>auth.required</code> .
2	Ordinary member → 403 on the admin ledger	The one that matters. Detail below.
3	The real sign-in journey completes	Phone → code → wallet. Asserts the balance renders as <code>0 XOF</code> and not <code>0,00 XOF</code> , since XOF has no minor unit and the wrong exponent would be a 100× error. Also asserts no wording on the screen calls the balance a deposit or protected funds, and that signed out, the same route redirects.

Why assertion 2 checks three things before it checks the 403

A 403 on its own proves nothing. A broken session, an empty cookie jar, or a route that refuses everyone would all produce one. So before asserting the denial, the test establishes that the session is genuinely live and that the actor genuinely lacks the permission:

STEP	EXPECTED	WHY IT IS THERE
<code>/api/v1/me</code>	200	The session resolves to a real actor. Without this the 403 could be a dead session.
permissions contain <code>wallet.read_own</code>	true	The actor really was provisioned and granted <code>member</code> .
permissions contain <code>ledger.read</code>	false	The permission the admin route needs is one this actor genuinely does not hold.
<code>/api/v1/admin/ledger</code>	403	The assertion. Code <code>authz.denied</code> , message naming the permission.
<code>/api/v1/wallet</code>	200	The 403 is an authorization decision, not a blanket refusal.

This shape is a direct consequence of Phase 1: two tests there passed while actually relying on unrelated mechanisms, and were only found by mutating the code they claimed to protect.

Two implementation decisions worth checking me on

The API calls run inside the page, not through Playwright's request context

The session cookie is marked `Secure` — the application sets that whenever `NODE_ENV` is production, which `next start` is. Playwright's own request context declines to attach a Secure cookie over `http`, so it reported 401 for a perfectly valid session.

I only noticed because the test asserts the session is live *first*. Without that step I would have had a green 403 that proved nothing at all. The calls now run as `fetch` inside the page, which uses the browser's real cookie jar on a real origin — closer to what the application actually faces, not further from it.

The sign-in test reads the code from the sender, never from the database

Since review decision 1 the verification code exists only as a keyed hash in `otp_challenges`. So the test reads what `ConsoleSender` prints — the same channel a developer uses — and no test-only code path was added to make this easier.

A PROPERTY WORTH NOTICING

A browser test that could look the verification code up in PostgreSQL would itself be evidence that the Phase 1 correction had been undone. The awkwardness here is the security guarantee working.

Three tests passing proves nothing on its own. Each guarantee was broken on purpose to see whether its test noticed.

DELIBERATE DEFECT	RESULT	WHICH TEST
Admin route stops calling <code>authorize()</code>	CAUGHT	Assertion 2 fails; 1 and 3 still pass.
<code>requireActor()</code> invents an actor instead of refusing	CAUGHT	Assertion 1 fails; 2 and 3 still pass.
OTP verification accepts any code	NOT CAUGHT HERE	None of the three. See below.

THE THIRD ONE IS THE HONEST RESULT, NOT A GAP

The browser tests always submit the *right* code, so they are structurally incapable of seeing that property — and they should not be credited with it. I verified where it does live rather than assuming: the same mutation fails **seven tests** in `packages/core/src/auth`, including refuses a wrong code, the attempt-limit test, expiry, cross-number replay, the identical-failure test, and the audit test.

The layering is right — browser tests own authorization, the core suite owns OTP semantics — but a reader of this document should know exactly which of the two is carrying which guarantee.

Run #9 on commit `136d330`, three jobs, all green. The browser job runs in parallel with the others on its own PostgreSQL service, provisioned from zero, so neither can mask the other.

JOB	RESULT	DURATION
browser · authorization assertions	SUCCESS	1m 48s
lint · typecheck · test	SUCCESS	1m 40s
dependency scan	SUCCESS	23s

The browser job's steps: provision the database · install the browser · build the web app · run the assertions. The report-upload step correctly shows skipped, since it only runs on failure. On failure it attaches the Playwright trace, screenshots and the server log.

Local verification on the same commit, for completeness:

```
pnpm install --frozen-lockfile    exit 0
db reset → migrate → seed        from zero
pnpm lint                         exit 0
pnpm typecheck                   exit 0
pnpm test                        172 passed | 3 skipped (16 files)
pnpm build                       exit 0
concurrency suite                 6 passed
reconcile-check                  clean, zero drift
pnpm audit --audit-level high    No known vulnerabilities found
playwright test                   3 passed
```

FILE	LINES	WHAT
apps/web/e2e/authorization.spec.ts	+164	New. The three assertions. This is the file to read.
.github/workflows/ci.yml	+77	The <code>browser</code> job: own PostgreSQL service, own from-zero provisioning, browser install, production build, the tests.
apps/web/playwright.config.ts	+60	New. Serves a production build on port 3100; never reuses a server someone left running, since it could be a different build.
pnpm-lock.yaml	+51/-7	<code>@playwright/test</code> at a pinned 1.56.1.
.gitignore	+5	Playwright output.
apps/web/package.json	+3/-1	The dependency and a <code>test:e2e</code> script.

7

Risks, and what has not changed

ITEM	STATE
<code>playwright install --with-deps chromium</code> had never run on a GitHub runner from this repository	NOW CLOSED — it worked, in 41s, the job's slowest step
Coverage is three assertions, not a suite. A UI regression outside these paths still ships green.	ACCEPTED, DELIBERATELY
The browser job adds roughly two minutes to a run	NOTED
Local-only: <code>next build</code> fails with a spurious <code>useContext</code> error when <code>.next</code> is stale after a dependency change; <code>rm -rf apps/web/.next</code> fixes it. CI always builds fresh, so it cannot occur there.	NOTED

Unchanged and unweakened by this milestone: the OTP security model, Better Auth's absent phone plugin, the ledger and its database invariants, the authorization model, scoped RBAC, consent versioning, and every standing rule from the Phase 1 packet. No application file appears in the diff.

Next: milestone 2 — stores and digital products. A seller creates a store and publishes a paid digital product at a public URL; a visitor reaches it and sees its price. Checkout, consent enforcement and the withdrawal path are separate later milestones. **Not started.**